NPU 快速入门

开发指南

版本号: 2.0

发布日期: 2025.10.16

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2024.03.12	AWA1382	初始版本
1.1	2024.04.11	AWA1382	增加 MR536、T536 平台说明
1.2	2024.11.13	AWA1382	1) 增加 A733 平台说明； 2) 更新 T527 平台说明； 3) 增加 SDK 产品包介绍一章。
1.3	2025.04.03	AWA1382	**全部章节** 增加 T736 平台说明。 **第四章 部署示例程序** 更新 env.sh 使用和更新 Linux 编译步骤。
2.0	2025.10.16	AWA1382	**全部章节** 根据 model zoo 的使用更新说明。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**

1.1 文档简介 1

1.2 目标读者 1

1.3 适用范围 1

1.4 文档约定 1

1.4.1 名词解释 1

1.4.2 标志说明 2

2 SDK 产品包介绍 **3**

2.1 常用工具 3

2.2 Model Zoo 3

2.3 参考文档 4

2.4 驱动与中间件 5

3 开发环境准备与检查 **6**

4 部署示例程序 **7**

4.1 进入 Docker 容器 7

4.2 模型转换量化 8

4.3 编译示例程序 11

4.3.1 解压 opencv 压缩包 11

4.3.2 准备交叉编译工具链 12

4.3.2.1 Linux 12

4.3.2.2 Android 12

4.3.3 编译和运行 12

4.3.3.1 Linux 12

4.3.3.2 Android 13

4.3.4 查看结果 14

5 常见问题 **16**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文档对 NPU SDK 产品包进行介绍，以及面向零基础用户介绍如何快速在 PC 上完成模型转换，并部署到板端。

1.2 目标读者

本文档（本指南）主要适用于以下人员：

- 技术支持工程师
- AI 软件开发工程师
- AI 算法开发工程师

1.3 适用范围

硬件平台：V85x、R853、MR527、T527、AI985、MR536、T536、A733、T736

软件平台：Tina 系统、Android13 及以上系统

1.4 文档约定

1.4.1 名词解释

术语	解释说明
NPU	Neural Network Processing Unit，神经网络处理器。
NBG	Network Binary Graph，驱动可识别的网络二进制文件。
pcq	perchannel quantize，逐通道量化。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4.2 标志说明

本文中出现的符号如下：

⚠ 警告

- 提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

注意

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

广州星翼电子科技有限公司zdyzlinux

2 SDK 产品包介绍

本节简要介绍 SDK 产品包的组成及使用方法。包括如下内容：

类型	获取方式
工具	在 全志客户服务平台 的资源下载，下载页面详见《NPU_环境部署_参考指南》。
Model Zoo	在 全志客户服务平台 的资源下载，下载页面详见《NPU_环境部署_参考指南》。
文档	在 Model Zoo 的 docs 目录下，或从 全志客户服务平台 的文档中心中获取。
驱动源码	Linux5.4 及以上：位于系统 SDK，Android/longan/bsp/drivers/npu 或 Tina/bsp/driver/npu； Linux4.9：位于系统 SDK，lichee/linux-4.9/drivers/char/sunxi_nna_xxx
中间件	在 Model Zoo 的 common/npuruntime 目录下，或从系统 SDK 的 ai-sdk 中获取。

注意

不同平台的 *ai-sdk* 仓库在不同位置：
V85x/R853：*SDK* 中不包含 *ai-sdk* 目录，中间件位于 *SDK*`package/allwinner/libsdk-viplite-driver/sdk_release`目录。
MR527/AI985：位于 *SDK*，`openwrt/package/allwinner/vision/ai-sdk`。
MR536：位于 *SDK*，`platform/allwinner/vision/ai-sdk`（编译在`openwrt/package/allwinner/vision/ai-sdk`进行）。
T527/T536/T736 Linux：位于 *SDK*，`platform/allwinner/vision/ai-sdk`（编译在`buildroot/buildroot-xxxxxx`进行）。
T527/A733 Android：位于 *SDK*，`vendor/aw/ai-sdk`。

2.1 常用工具

NPU 开发工具主要包括以下几个：

- **Acuity Toolkit**：模型转换工具，用于模型导入、优化、量化、推理以及导出等。
- **VivanteIDE**：IDE 仿真工具，用于 PC 侧的模型仿真验证和 Profile 性能仿真分析。
- **NBInfo**：NBG 分析工具，用于 PC 端查看分析转换后的驱动可识别的模型文件。
- **Docker 镜像包**：集成平台部署 AI 算法所需的软件、工具包，即包含上述三个工具。
- **vpm_run**：模型运行工具，用于端侧运行任意模型的通用程序。
- **Model Zoo**：提供主流算法的部署示例。

2.2 Model Zoo

AWNPU Model Zoo 中主要包含两种示例：常用模型的例子和功能性使用的示例。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

常用模型的例子包含模型转换和板端 Demo，放在`awnpu_model_zoo/examples`目录，主要有以下示例：

功能分类	例程	备注
图像分类	mobilenetv2	
图像分类	resnet50v2	
目标检测	yolov5	
目标检测	yolov8	
人体关键点	yolov8_pose	
图像分割	yolov8_seg	

功能性使用的示例主要包含板端 Demo，部分也包含模型转换，放在`awnpu_model_zoo/functions`目录，主要有以下示例：

功能分类	例程	备注
模型通用测试程序	vpm_run	
多线程程序	multi_thread	
共享运行内存程序	share_memory_pool	
多输入模型程序	multi_input	

2.3 参考文档

目前提供的指导文档、说明文档汇总如下：

功能分类	文档	内容	位置
环境搭建	NPU_开发环境搭建_参考指南	指导安装 NPU 工具	全志客户服务平台或 Model Zoo 的 docs
模型部署	NPU_快速入门_开发指南	介绍产品包，指导快速跑通示例和端侧程序	全志客户服务平台或 Model Zoo 的 docs
模型部署	NPU_模型部署_开发指南	介绍系统及详细说明模型部署、工具使用	全志客户服务平台或 Model Zoo 的 docs
模型部署	NPU_算子支持列表	提供算子支持列表及各框架支持情况	全志客户服务平台或 Model Zoo 的 docs
模型部署	NPU_RuntimeAPI_参考指南	详细说明端侧应用程序的 API	全志客户服务平台或 Model Zoo 的 docs

版权所有 © 珠海全志科技股份有限公司。保留一切权利

功能分类	文档	内容	位置
NPU 测试	[型号]_NPU_常见网络性能_测试报告	常见模型性能参考	全志客户服务平台

2.4 驱动与中间件

NPU 开发可以参考《NPU_模型部署_开发指南》的《NPU 驱动配置》和《驱动问题》，共有两套 NPU 模块驱动：

- aw_nna_vip/sunxi_nna_vip：VIPLite 版本 NPU 驱动。
- aw_nna_galcore/sunxi_nna_galcore：Unified 版本 NPU 驱动。

NPU 模块运行例程依赖中间件，其中：

- ai-sdk/viplite-android：Android 环境 VIPLite 版本 NPU 中间件。
- ai-sdk/viplite-tina：Linux 环境 VIPLite 版本 NPU 中间件。
- ai-sdk/unified-android：Android 环境 Unified 版本 NPU 中间件。
- ai-sdk/unified-tina：Linux 环境 Unified 版本 NPU 中间件。

注意

VIPLite 版本的中间件，在 *v2.0.0* 之前，包含 *libVIPlite.so* 和 *libVIPuser.so*。*v2.0.0* 之后，包含 *libNBGlinker.so* 和 *libVIPhal.so*。库名字有所差异，接口 *API* 基本不变，*API* 具体用法可参考《*NPU_RuntimeAPI*_参考指南》。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 开发环境准备与检查

根据《NPU_开发环境部署_参考指南》，部署 PC 侧的 ubuntu 环境，使用 Docker 镜像环境或自行安装两个工具均可，以及获取 Model Zoo 压缩包。

首先，使用`pegasus --help`命令检查 Acuity Toolkit 是否可用，打印关键信息如下：

```
Pegasus commands.

positional arguments:
  {import,export,generate,prune,inference,quantize,train,dump,measure,help}
    import              Import models.
    export              Export models.
    generate            Generate metas.
    prune               prune models.
    inference           Inference model and get result.
    quantize            Quantize model.
    train               Train model.
    dump                Dump model activations.
    measure             Get amount of calculation, parameter and activation.
    help                Print a synopsis and a list of commands.

optional arguments:
  -h, --help            show this help message and exit
```

然后，使用`echo`命令检查 ACUITY_PATH 环境变量是否配置，方法如下：

```
~/awnpu_model_zoo$ echo ${ACUITY_PATH}
/root/Verisilicon_Tool_Acuity_Toolkit/acuity-toolkit-binary-x.x.x/bin/
```

以及，使用`echo`命令检查 VIV_SDK 环境变量是否配置，方法如下：

```
~/awnpu_model_zoo$echo ${VIV_SDK}
/root/Vivante_IDE/VivanteIDEx.x.x/cmdtools
```

本文所用示例从 Model Zoo 中获取。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 部署示例程序

本章将介绍如何快速在开发板上运行示例程序。

4.1 进入 Docker 容器

在 Ubuntu 机器操作进入 Docker 容器

```
sudo docker run -it --privileged -v /home/${USER}/docker_data:/workspace ubuntu-npu:v1.x /bin
    /bash

#或者使用容器ID进入
sudo docker ps -a
sudo docker exec -it 容器ID /bin/bash
```

将 awnpu_model_zoo 放在 docker_data 目录（该目录已挂载到 docker 容器）。

```
root@18cb14984dc0:/workspace/awnpu_model_zoo# tree -L 2
.
|-- 0-toolchains                #存放各平台的交叉编译工具链
|   |-- android-ndk-r25b-linux.zip
|   |-- arm-openwrt-linux-muslgnueabi.zip
|   `-- gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz
|-- 3rdparty                    #存放第三方库，例如opencv库
|   `-- opencv
|-- cmake_toolchain             #交叉编译工具链路径设置
|   |-- android_ndk_build_env.sh
|   |-- compiler.toolchain-linux-aarch64.cmake
|   `-- compiler.toolchain-linux-armhf.cmake
|-- common                      #存放NPU的调用接口代码，中间件so库
|   `-- npuruntime
|-- doc                         #文档
|-- examples                    #常用模型的例子，例如Mobilenetv2、yolov5、yolov8等
|   |-- dataset
|   |-- mobilenetv2
|   |-- resnet50v2
|   |-- yolov5
|   |-- yolov8
|   |-- yolov8_hybrid
|   `-- yolov8_pose
|-- functions                   #功能性使用的示例，如多线程、共享运行内存示例、模型通用测试示例
|   |-- multi_input
|   |-- multi_thread
|   |-- share_memory_pool
|   `-- vpm_run
|-- scripts_model_convert       #存放模型转换脚本
|   |-- pegasus_dump.sh
|   |-- pegasus_export_ovx_nbg-hybrid.sh
|   |-- pegasus_export_ovx_nbg.sh
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
|    |-- pegasus_import.sh
|    |-- pegasus_inference.sh
|    |-- pegasus_quantize.sh
|    `-- pegasus_quantize_hybrid.sh
|-- version                          #版本号
`-- README.md
```

4.2 模型转换量化

在awnpu_model_zoo/examples中，有一些常用模型的转换示例。下面以 yolov5 模型为例，介绍部署脚本的使用方法。

examples/yolov5 的目录结构如下：

```
~/docker_data/awnpu_model_zoo/examples$ tree
.
├── dataset                                    #用于量化校正的数据集
│   ├── coco_12
│   │   ├── COCO_train2014_000000000081.jpg
│   │   ├── COCO_train2014_000000001569.jpg
│   │   ├── COCO_train2014_000000002849.jpg
│   │   ├── COCO_train2014_000000004139.jpg
│   │   ├── COCO_train2014_000000005933.jpg
│   │   ├── COCO_train2014_000000006338.jpg
│   │   ├── COCO_train2014_000000361332.jpg
│   │   ├── COCO_train2014_000000363483.jpg
│   │   ├── COCO_train2014_000000366396.jpg
│   │   ├── COCO_train2014_000000376972.jpg
│   │   ├── COCO_train2014_000000500390.jpg
│   │   ├── dataset.txt                        #记录量化校正的所有数据的路径
│   │   ├── dog.jpg
│   │   └── dog_ori.jpg
│   └── imagenet_10
├── yolov5
│   ├── CMakeLists.txt
│   ├── convert_model
│   │   ├── config_yml.py                      #用于配置通道mean和scale等
│   │   ├── convert_model_env.sh
│   │   ├── python
│   │   │   ├── onnx_extract.py
│   │   │   └── yolov5s.onnx                   #yolov5s的onnx模型文件，已把输入尺寸固定为
    (1,3,640,640)
│   │   └── yolov5s_rt.onnx                    #处理后用于部署的onnx模型文件
│   ├── main.cpp
│   ├── model
│   │   ├── dog.jpg
│   │   ├── yolov5s_rt_uint8_a733.nb
│   │   └── yolov5s_rt_uint8_t527.nb
│   ├── model_config.h
│   ├── README.md
│   ├── yolov5_post.cpp
│   └── yolov5_pre.cpp
└── yolov8_seg
```

固定 shape 的模型生成、命令详细解释、多输入模型的配置等请参考《NPU_模型部署_开发指南》的《NPU 使用说明》一章。若量化后精度不满足需求，均需要手动调整量化描述文件进行精度优化，请参考《NPU_模型部

版权所有 © 珠海全志科技股份有限公司。保留一切权利

署_开发指南》的《量化精度调优》一章。

部署脚本放在awnpu_model_zoo/scripts_model_convert目录中。

```
~/docker_data/awnpu_model_zoo/scripts_model_convert$ tree
.
├── pegasus_dump.sh
├── pegasus_export_ovx_nbg-hybrid.sh
├── pegasus_export_ovx_nbg.sh
├── pegasus_import.sh
├── pegasus_inference-hybrid.sh
├── pegasus_inference.sh
├── pegasus_quantize-hybrid.sh
└── pegasus_quantize.sh
```

首先，进入convert_model目录；

```
cd ./convert_model/
```

检查或修改 config_yml.py 文件的相关参数配置。

```
# "database"
DATASET = '../../dataset/coco_12/dataset.txt'
DATASET_TYPE = "TEXT"
# mean, scale
MEAN    = [0, 0, 0]
SCALE   = [0.0039216, 0.0039216, 0.0039216]

# reverse_channel: True bgr, False rgb
REVERSE_CHANNEL = False
# add_preproc_node, True or False
ADD_PREPROC_NODE = True
# "preproc_type"
PREPROC_TYPE = "IMAGE_RGB"
# add_postproc_node, quant output -> float32 output
ADD_POSTPROC_NODE = True
```

然后使用 convert_model_env.sh 为部署脚本生成软连接，目录内容变成如下：

```
# using xxx_env.sh to create softlink
./convert_model_env.sh

root@4a4399e1bb2d:/workspace/awnpu_model_zoo/examples/yolov5/convert_model# tree
.
|-- config_yml.py
|-- convert_model_env.sh
|-- pegasus_export_ovx_nbg.sh -> ../../../scripts_model_convert/pegasus_export_ovx_nbg.sh
|-- pegasus_import.sh -> ../../../scripts_model_convert/pegasus_import.sh
|-- pegasus_inference.sh -> ../../../scripts_model_convert/pegasus_inference.sh
|-- pegasus_quantize.sh -> ../../../scripts_model_convert/pegasus_quantize.sh
|-- python
|   |-- onnx_extract.py
|   `-- yolov5s.onnx
`-- yolov5s_rt.onnx
```

按照以下步骤进行，生成 uint8 量化的 NBG 文件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
# 导入
# pegasus_import.sh <model_name>
./pegasus_import.sh yolov5s_rt

# 量化
# pegasus_quantize.sh <model_name> <quantize_type> <calibration_set_size>
./pegasus_quantize.sh yolov5s_rt uint8 12

# 仿真（可选）
# pegasus_inference.sh <model_name> <quantize_type>
./pegasus_inference.sh yolov5s_rt uint8

# 导出nb模型
# pegasus_export_ovx_nbg.sh <model_name> <quantize_type> <platform>
./pegasus_export_ovx_nbg.sh yolov5s_rt uint8 t527
```

导出的模型文件存放在../model 目录，生成的文件是 yolov5s_rt_uint8_t527.nb。

步骤简要说明：

1. convert_model/python/yolov5s.onnx 文件，是基于官方的 yolov5s.onnx，把输入尺寸固定为 (1,3,640,640)；
2. convert_model/yolov5s_rt.onnx 文件，是基于上述 yolov5s.onnx，运行 onnx_extract.py，移除 Transpose 节点；
3. config_yml.py 中，注意根据算法输入的预处理修改 mean，scale 的值，这里 yolov5s 的 scale 为 0.00392157；以及推理结果由 NPU 内部做反量化浮点输出；
4. `pegasus_import.sh 模型名称`，进行模型导入，转化出模型的结构、权重、输入配置、输出配置，并按照配置修改 yolov5s_rt_inputmeta.yml；
5. `pegasus_quantize.sh 模型名称 量化类型[uint8, int16, pcq] 校正集数量`，进行模型量化；
6. `pegasus_inference.sh 模型名称 量化类型`，进行模型推理；
7. `pegasus_export_ovx_nbg.sh 模型名称 量化类型 soc平台`，进行模型导出。

生成文件情况如下：

```
~/docker_data/awnpu_model_zoo/examples/yolov5/convert_model$ tree
.
├── config_yml.py
├── convert_model_env.sh
├── pegasus_export_ovx_nbg.sh -> ../../../scripts_model_convert/pegasus_export_ovx_nbg.sh
├── pegasus_import.sh -> ../../../scripts_model_convert/pegasus_import.sh
├── pegasus_inference.sh -> ../../../scripts_model_convert/pegasus_inference.sh
├── pegasus_quantize.sh -> ../../../scripts_model_convert/pegasus_quantize.sh
├── python
│   ├── onnx_extract.py
│   └── yolov5s.onnx
├── yolov5s_rt.data                    #网络权重文件
├── yolov5s_rt_inputmeta.yml            #输入描述文件
├── yolov5s_rt.json                    #网络结构文件
├── yolov5s_rt.onnx
├── yolov5s_rt_postprocess_file.yml     #输出描述文件
└── yolov5s_rt_uint8.quantize           #量化描述文件
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 编译示例程序

在awnpu_model_zoo/examples中，有不同模型的端侧示例程序，在awnpu_model_zoo/functions中，有功能性的端侧示例程序。提供的示例可以直接在 docker 环境下编译，需检查交叉编译工具链是否正确；其中awnpu_model_zoo/common/npuruntime/提供的中间件库作为编译链接使用，板端运行以板端的 so 库为准。

注意

不同的示例用法存在差异，请按照各自目录下 *README.md* 中的步骤操作。

下面以 yolov5 模型为例，介绍编译方法。examples/yolov5 的目录结构如下：

```
~/docker_data/awnpu_model_zoo/examples$ tree
.
├── build_android.sh     #android系统demo编译脚本
├── build_linux.sh       #linux系统demo编译脚本
├── yolov5
│   ├── CMakeLists.txt
│   ├── convert_model
│   ├── main.cpp         #NPU运行实现
│   ├── model
│   │   ├── dog.jpg      #输入数据
│   │   ├── yolov5s_rt_uint8_a733.nb
│   │   └── yolov5s_rt_uint8_t527.nb     #NBG模型，各平台不通用，请选择正确平台的NBG模型
│   ├── model_config.h
│   ├── yolov5_post.cpp      #后处理实现
│   └── yolov5_pre.cpp       #预处理实现
```

4.3.1 解压 opencv 压缩包

```
# 进入目录
cd ../../../3rdparty/opencv/
# 解压，选择对应平台
# armhf，例如：V85x，R853平台使用
unzip opencv-3.4.16-gnueabihf-linux.zip
# linux aarch64，例如：T527/MR527/MR536/T536/A733/T736 平台使用
unzip opencv-4.9.0-aarch64-linux-sunxi-glibc.zip
# android ，例如：T527/A733/T736 平台使用
unzip opencv-4.9.0-android.zip
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3.2 准备交叉编译工具链

4.3.2.1 Linux

```
# 进入目录
cd ../../0-toolchains/
# 解压，选择对应平台
# armhf，例如：V85x，R853平台使用
unzip arm-openwrt-linux-muslgnueabi.zip
chmod 777 -R ./arm-openwrt-linux-muslgnueabi
# aarch64，例如：T527/MR527/MR536/T536/A733/T736 平台使用
tar xvf gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz
```

编译脚本会根据平台自动选择交叉编译工具链，若需使用其它路径的工具链，可在`cmake_toolchain`目录修改`.cmake`文件内容指定对应的交叉编译工具链路径。

4.3.2.2 Android

下载 Android NDK，下载地址：https://developer.android.google.cn/ndk/downloads/index?hl=th

将下载的 NDK 放到编译机器目录，例如：./0-toolchains/；

请根据下载的版本修改`cmake_tool` 文件。

4.3.3 编译和运行

4.3.3.1 Linux

使用 build_linux.sh 脚本进行编译。用法如下：

```
# 用法：./build_linux.sh -t {TARGET_NAME} [-p {DEMO_PATH}] [-h]
# -t:  target，平台类型，例如 v853 / r853 / mr527 / t527 / mr536 / t536 / a733 / t736
# -p:  demo path，编译示例的路径，可选，例如 yolov5、vpm_run，默认值为当前目录（./）
# -h:  help，帮助说明

# 途径一：在yolov5目录编译
cd ../example/yolov5/
./../build_linux.sh -t <platform>
# 途径二：在example目录，再选择yolov5目录编译
cd ../example
./build_linux.sh -t <platform> -p yolov5
```

以下说明以 T527 平台为例；

```
cd ../example/yolov5/
./../build_linux.sh -t t527
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

编译完成后，会在 yolov5 目录下产生 install 文件夹，其中有编译好的可执行文件，以及测试图片等相关文件。参考目录结构如下：

```
~/docker_data/awnpu_model_zoo/examples/yolov5$ tree install/
install/
└── yolov5_demo_linux_t527  # t527平台
    ├── model               #存放模型、测试图片等文件
    │   ├── dog.jpg
    │   └── yolov5s_rt_uint8_t527.nb
    └── yolov5_demo_t527    #可执行文件
```

推送可执行文件、模型文件、输入图片到板端目录（建议推到 tf 卡目录/mnt/extsd，空间充足），建议推送整个 yolov5_demo_linux_t527 文件夹到板端。

```
adb push install\yolov5_demo_linux_t527 /mnt/UDISK/
```

执行以下命令，在板端运行可执行文件：

```
# 进入板端
adb shell

# 进入 yolov5_demo_linux_t527 目录
cd /mnt/UDISK/yolov5_demo_linux_t527

# 运行可执行文件
# ./yolov5_demo_t527 -h 查看执行示例说明
# 用法：./yolov5_demo_t527 -nb modle_path -i input_path [-l loop_run_count] [-m malloc_mbyte]
chmod +x ./yolov5_demo_t527
./yolov5_demo_t527 -nb model/yolov5s_rt_uint8_t527.nb -i model/dog.jpg
```

4.3.3.2 Android

使用 build_android.sh 脚本进行编译。用法如下：

```
# 用法: ./build_android.sh -t {TARGET_NAME} [-p {DEMO_PATH}] [-h]
# -t:  target, 平台类型，例如 t527 / a733 / t736
# -p:  demo path，编译示例的路径，可选，例如 yolov5、vpm_run，默认值为当前目录（./）
# -h:  help，帮助说明

# 途径一：在yolov5目录编译
cd ../example/yolov5/
./../build_android.sh -t <platform>
# 途径二：在example目录，再选择yolov5目录编译
cd ../example
./build_android.sh -t <platform> -p yolov5
```

以下说明以 T527 平台为例；

```
cd ../example/yolov5/
./../build_android.sh -t t527
```

编译完成后，会在 yolov5 目录下产生 install 文件夹，其中有编译好的可执行文件，以及测试图片等相关文件。参考目录结构如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
~/docker_data/awnpu_model_zoo/examples/yolov5$ tree install/
install/
└── yolov5_demo_android_t527    # t527平台
    ├── lib                     #依赖库
    │   ├── libomp.so
    │   ├── libVIPlite.so
    │   └── libVIPuser.so
    ├── model                   #存放模型、测试图片等文件
    │   ├── dog.jpg
    │   └── yolov5s_rt_uint8_t527.nb
    └── yolov5_demo_t527        #可执行文件
```

推送可执行文件、模型文件、输入图片到板端目录，建议推送整个 yolov5_demo_android_t527 文件夹到板端。

```
# 修改权限
adb root
adb remount

adb push install\yolov5_demo_android_t527 /data/local/tmp/
```

执行以下命令，在板端运行可执行文件：

```
# 进入板端
adb shell

# 进入 yolov5_demo_android_t527 目录
cd /data/local/tmp/yolov5_demo_android_t527

# 设置依赖库环境
export LD_LIBRARY_PATH=./lib

# 运行可执行文件
# ./yolov5_demo_t527 -h 查看执行示例说明
# 用法：./yolov5_demo_t527 -nb modle_path -i input_path [-l loop_run_count] [-m malloc_mbyte]
chmod +x ./yolov5_demo_t527
./yolov5_demo_t527 -nb model/yolov5s_rt_uint8_t527.nb -i model/dog.jpg
```

4.3.4 查看结果

以下说明以 T527 平台为例，运行打印输出分析如下：

```
t527-demo:/data/local/tmp/yolov5_demo_android_t527 #./yolov5_demo_t527 -nb model/
    yolov5s_rt_uint8_t527.nb -i model/dog.jpg
model_file=model/yolov5s_rt_uint8_t527.nb, input=model/dog.jpg, loop_count=1, malloc_mbyte=10
#输入信息
input  0 dim 3 640 640 1, data_format=2, quant_format=0, name=input[0], none-quant
#输出信息
output 0 dim 80 80 85 3, data_format=0, name=uid_20000_sub_uid_1_out_0, none-quant
output 1 dim 40 40 85 3, data_format=0, name=uid_20001_sub_uid_1_out_0, none-quant
output 2 dim 20 20 85 3, data_format=0, name=uid_20002_sub_uid_1_out_0, none-quant
nbg name=model/yolov5s_rt_uint8_t527.nb, size: 6398208.
create network 0: 23015 us.
prepare network: 12282 us.
buffer ptr: 0xb40000797f83b140, buffer size: 1228800
feed input cost: 24620 us.
network: 0, loop count: 1
run time for this network 0: 51030 us.
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
#检测结果
detection num: 3
16:  92%, [ 137,  223,  307,  534], dog
 2:  65%, [ 467,   75,  692,  170], car
 1:  58%, [ 162,  129,  567,  416], bicycle
draw objects time : 64 ms
destory npu finished.
~NpuUint.
```

默认情况下，输出图片保存路径为`yolov5_demo_android_t527/output_yolov5.png`，可以通过 adb 工具从板端拉取到本地查看，执行一下命令：

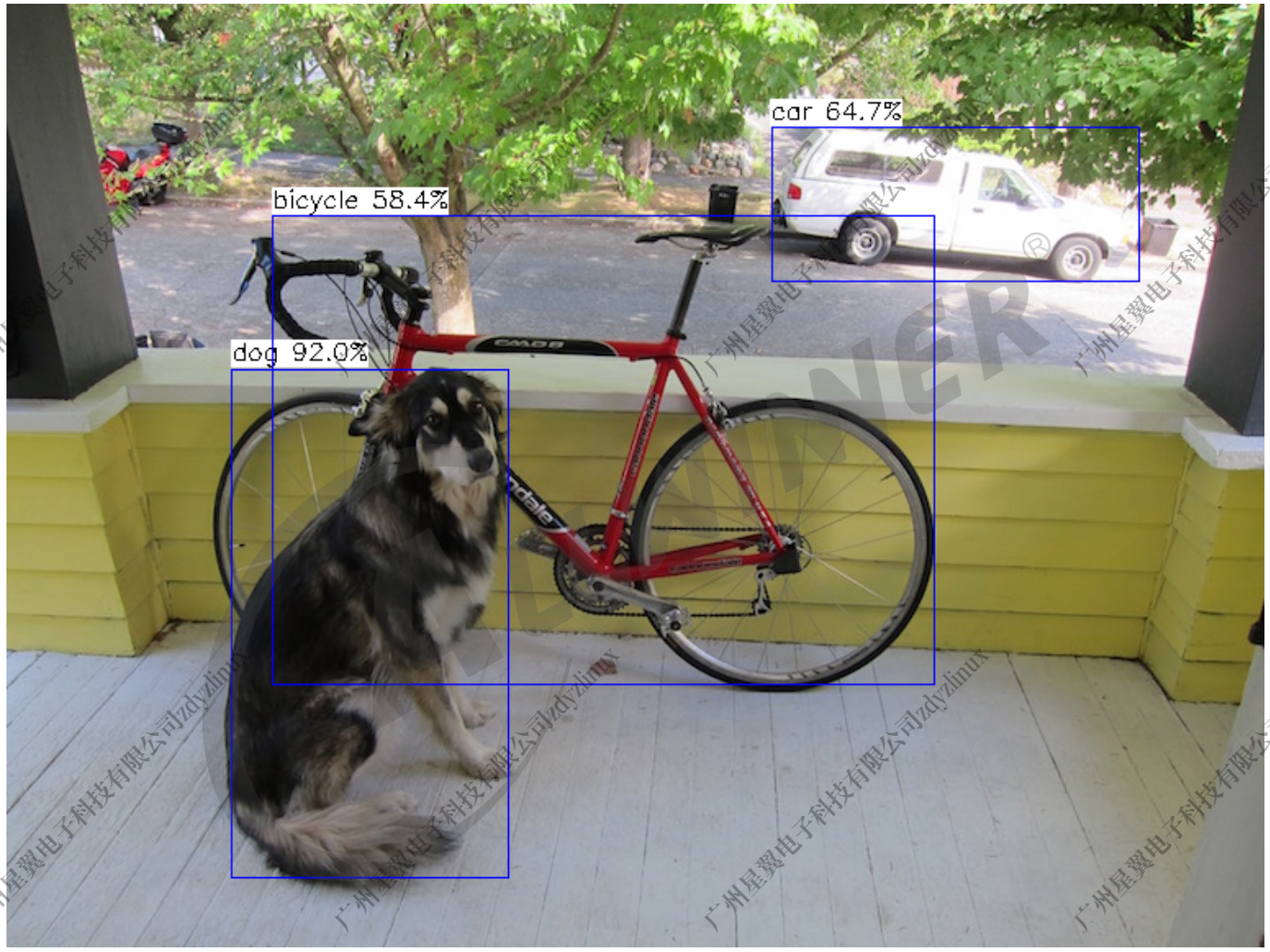

图 4-1: yolov5 输出图片

广州星翼电子科技有限公司zdyzlinux

5 常见问题

1）yolov5 检测结果为 0

```
#检测数量为0
detection num: 0
destory npu finished.
```

可能原因：NBG 模型文件的输出与运行程序不一致，运行程序是 3 个输出。看以下打印可检查，可见 NBG 模型文件是四个输出。

```
output 0 dim 85 25200 1, data_format=2, name=uid_7_out_0, elements=2142000, scale=2.498428,
    zero_point=0
create output buffer 0: 2142000
output 1 dim 85 80 80 3 1, data_format=2, name=uid_6_out_0, elements=1632000, scale=0.085724,
     zero_point=210
create output buffer 1: 1632000
output 2 dim 85 40 40 3 1, data_format=2, name=uid_5_out_0, elements=408000, scale=0.071622,
    zero_point=204
create output buffer 2: 408000
output 3 dim 85 20 20 3 1, data_format=2, name=uid_4_out_0, elements=102000, scale=0.071997,
    zero_point=196
create output buffer 3: 102000
```

解决方法：检查 yolov5s_xxx.onnx 文件的输出是否匹配，可参考 convert_model/python/onnx_extract.py 进行裁剪。

2）yolov5 检测结果混乱

```
detection num: 921
32: 100%, [ 308,   37,  312,   38], sports ball
50: 100%, [ 173,   18,  173,   27], broccoli
50: 100%, [ 172,   20,  173,   26], broccoli
50: 100%, [ 620,   34,  621,   40], broccoli
50: 100%, [ 172,   18,  173,   27], broccoli
```

可能原因：scale 未调整，量化结果不对，导致结果误差大。

解决方法：检查 yolov5sxxx_inputmeta.yml 的 scale 是否为 0.00392157，否则请修正。

3）运行报错

```
./yolov5_demo_t527: line 1: syntax error: unterminated quoted string
```

可能原因：编译运行程序的工具链和端侧设备的工具链不一致。

解决方法：检查 CMakeLists.txt

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。